

Designing Large-scale AI Systems: CinePal, a Conversational Movie Recommender

Davide Donà Andrea Blushi

University of Trento

May 2026



Contents

1 Problem Statement

2 Architecture

3 Evaluation

4 Sprint

Problem Statement

- Standard movie recommenders optimise a **fixed signal** (clicks, ratings, watch-time).
- Our system has **no fixed objective**, following a conversational clustering paradigm.
- The **recommendation is the clustering**: the system proposes a final cluster of movies as the recommendation, and the conversation converges when the user accepts it.
- Dataset: **~45,000 movies** (The Movies Dataset, extracted from TMDB).

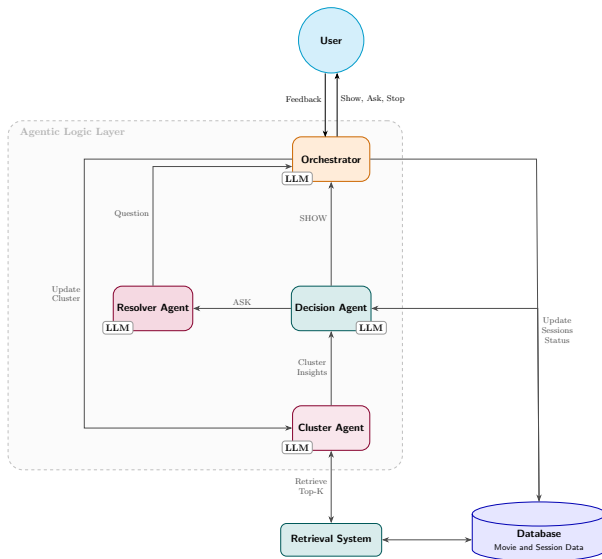
How Conversational Clustering Applies Here?

- **Retrieval:** user query is used to retrieve a set of candidate movies from the catalogue
- **Clustering:** candidates are soft-clustered into multiple named and described groups.
- **Decision** chooses **Show**, **Ask** or **Stop** based on cluster uncertainty and session state.

Challenges

- **Stopping signal:** every turn has a cost, both the system and the user are incentivised to converge in fewer turns.
- **Satisfaction:** there is no ground-truth label, quality is measured only by whether the user accepts the final cluster.
- **Preference drift:** users may change their minds mid-session, and the system must adapt without feeling erratic or untrustworthy.
- **Cognitive load:** presenting clusters and asking questions must be done in a way that doesn't overwhelm or confuse users, especially as the number of clusters grows.
- **Cluster stability:** names must remain stable enough to feel familiar across turns yet update faithfully when splits or merges materially change a cluster's contents.

Architecture Diagram



Orchestrator

- Coordinates all agent handoffs
- Maintains the state of the conversation and builds the user preferences profile.
- Single writer to database, and to communicate with the user interface
- Handles conversation convergence and termination

Cluster Agent

- At the first turn or conversation drift, it retrieves new clusters.
- Embed the query, enriched with LLM-generated context, and performs vector similarity search to retrieve top-K films.
- Performs soft clustering using HDBSCAN on retrieved films' embeddings to identify a first set of clusters.
- If exists already a cluster, uses the user feedback to update the cluster assignment through an LLM-based approach
- LLM-assisted cluster title and description generation

Decision Agent

- Determines next action: **Show** or **Ask**
- It bases its decision on the current cluster distribution, and the user preferences profile
- Measures entropy to detect high divergence requiring more signal
- Identifies dominant cluster with high relevance for recommendation

Ambiguity Resolver

- Activated only when Decision Agent chooses **Ask**
- Generates targeted questions identifying specific attributes or preferences to clarify user intent

Our research question

Does conversational refinement converge to an user-accepted recommendation **faster** and with **higher satisfaction** than a one-shot vector search baseline?

Different Decision Approaches

We will compare the following Decision Agent strategies:

- **Uncertainty-driven:** ask about the title with the smallest gap between its top-two soft-assignment scores, maximally uncertain, most informative
- **Random:** select uniformly from all titles with low confidence
- **Boundary-driven:** select the title geometrically nearest to the midpoint between the two highest-weighted cluster centroids
- **Popularity-first:** among ambiguous titles with low confidence, ask about the one with the highest vote count

Evaluation Metrics

- **Turns to convergence:** oracle turns from start to acceptance signal
- **Cognitive load per turn:** titles shown + clusters shown + question complexity, averaged per session
- **Oracle satisfaction rate:** fraction of sessions converging before the turn budget is exhausted
- **Drift detection rate:** fraction of injected contradictions correctly surfaced
- **LLM-as-Judge scores:** clustering coherence, question quality, preference profile fidelity
- All quantitative claims carry **95% bootstrap confidence intervals**

Oracle Satisfaction

No external ground truth exists, Three complementary signals are used:

- **Explicit acceptance:** oracle says "*perfect*" / "*yes, that's it*", or gives no corrective feedback for 2 consecutive turns
- **Pairwise probe:** after convergence, sample pairs from the accepted cluster and ask "*do these two belong together?*" — validates internal coherence without any external label
- **Hidden-spec overlap** (simulated oracles only): each persona has a pre-written preference spec (genres, styles, exclusions); titles in the accepted cluster are scored against it — metric: fraction of titles satisfying all positive and no negative criteria

What We Did

- Drafted comprehensive project documentation
- Implemented core database schema (Davide)
- Built ingestion pipeline for movie metadata and embeddings (Davide)
- Built retrieval system with vector-based movie search (Davide)
- Developed Cluster Agent (Davide)
- Developed Orchestrator, Decision, Ambiguity (Andrea)
- Created minimal React frontend (Andrea)

What We Are Going To Do

- **Complete the architecture:** several components are still stubs or missing entirely, in order to have a improved system for evaluation.
- **Prompt engineering:** MVP prompts were kept as-is once functional, but they are very basic and likely suboptimal.
- **Responsiveness:** parallelise LLM calls as they are a major bottleneck, considering smaller models for less critical tasks.
- **Evaluation setup:** build LLM Oracle, LLM Judge and structure experimentation framework
- **Frontend & dashboard:**
 - Show representative movies alongside disambiguation questions to help users better understand and answer them;
 - Add message loading indicators to hide LLM latency and improve perceived responsiveness;
 - Build an admin panel to monitor through a dashboard experiment progress and quality metrics;